

Vết

Langfuse-class LLM observability, plus end-to-end tracing of Vietnamese AI routes (Zalo, FPT.AI, Viettel, Lark, Google Chat, .NET). Built-in generation layers for Amazon Bedrock, Google Cloud Vertex AI, and Microsoft Foundry.

Working name only. Replace before public launch. Vietnamese for “trace / mark.”

Status: Plan for implementation. Not a fork of Langfuse. Reuse MIT UI shell + rebrand.

Audience: Builder implementing tonight after reading this document.

Date: 9 September 2026 · updated same day (hyperscaler providers)

Related notes: docs/ideas/01-excel-pipeline-eval.md (parked), docs/ideas/02-agent-trace-usage-saas.md (absorbed as turn receipts).

License reminder: Langfuse web UI outside ee/ is MIT. Keep copyright notices. Do not use the Langfuse name or logo. Do not copy ee/ code.

Contents

1. One-page pitch
2. Decisions already made
3. Goals, non-goals, parked ideas
4. Legal: copying Langfuse UI
5. How the app looks
6. Information architecture
7. Part A — Langfuse-class core
8. Part A2 — Hyperscaler LLM layers (Bedrock, Vertex, Foundry)
9. Part B — Vietnamese routes & channels
10. Canonical data model
11. Ingest modes
12. Architecture & repo layout
13. APIs
14. SDKs
15. Security & PII
16. Implementation sequence (tonight → later)
17. Acceptance checks
18. Risks & kill criteria
19. Appendix — connector notes

1. One-page pitch

Langfuse shows messages.create. A Vietnamese production bot shows:

```
Zalo user → OA webhook → FPT.AI / Viettel NLP → LLM → TTS → reply on Zalo
```

If we only copy Langfuse, that path is invisible. **Vết** is LLM observability *and* channel observability.

Pitch: Langfuse for Vietnamese production bots — trace the Zalo / FPT / Viettel / .NET path, not only the LLM call.

Layer	Job
A. Core	Projects, traces, sessions, scores, prompts, playground, evals, usage dashboard — Langfuse-shaped.
A2. Hyperscalers	Built-in adapters: Amazon Bedrock, Google Cloud Vertex AI, Microsoft Foundry (plus Anthropic direct, OpenAI-compatible, FPT Factory).
B. Routes	Zalo OA/Bot, FPT.AI Conversation, Viettel ASR/TTS/NLP, Lark, Google Chat, .NET/Teams inbound.

A without B is a weaker Langfuse. B without A is a webhook logger. Both are required.

2. Decisions already made

Topic	Decision
Fork Langfuse monorepo?	No. Do not take ClickHouse/worker/ee/ as the product. Rebuild the useful subset.
Copy Langfuse UI layout?	Yes. Reuse MIT web/ shell (sidebar, tables, filters, trace waterfall, playground). Swap logo, name, colors, Vietnamese copy.
Pixel-clone from screenshots?	No. Use their MIT source for the shell, or rebuild the same IA with shadcn. Prefer MIT source for traces/playground speed.
Vietnamese UI?	Default vi , English toggle. Native strings, not browser translate.
Anthropic?	First-class wrap of @anthropic-ai/sdk / Python Anthropic client (not OpenInference-only).
Hyperscaler LLMs?	Yes — built-in. Amazon Bedrock, Google Cloud Vertex AI, Microsoft Foundry as named LLM connections + SDK wraps. Not “paste a custom base URL.”
Primary VN pair	Zalo OA tap + FPT.AI Conversation, then .NET SDK, then Viettel speech/NLP, then Lark + Google Chat.
Excel eval kit	Parked. Not in this build.

3. Goals, non-goals, parked ideas

Goals

- A developer can open a Zalo user session and see inbound message, NLU, LLM, tools, TTS, outbound reply as one tree.
- A developer can wrap Anthropic, Bedrock, Vertex, Foundry, and OpenAI-compatible / FPT Factory and see Langfuse-style generations tagged with provider + region/project.
- A .NET bot can emit OTLP and land in the same UI.
- Empty, loading, and error states exist in Vietnamese.
- The product does not claim to be Langfuse.

Non-goals (v1)

- Langfuse Cloud parity (annotation queues at their depth, prompt experiments at scale, 20 judge templates).
- ClickHouse as day-one store.
- Using Langfuse ee/ features (SCIM, their custom-logo Cloud path, etc.).
- Viettel Cyberbot as a public webhook (enterprise / sales API — SDK spans only until a spec exists).
- Zalo/FPT as “Playground model dropdowns” only — they are *channels*, not just LLM connections.
- Facebook Messenger / Viber / Telegram in v1 (same adapter interface later).

Parked

- Claude-for-Excel pipeline-point eval kit.
- Standalone “usage report SaaS” — the **turn receipt** in session detail covers this.

4. Legal: copying Langfuse UI

Langfuse LICENSE: content outside ee/, web/src/ee/, worker/src/ee/ is MIT. Permission to copy, modify, sell. Keep the copyright notice and permission text in distributions.

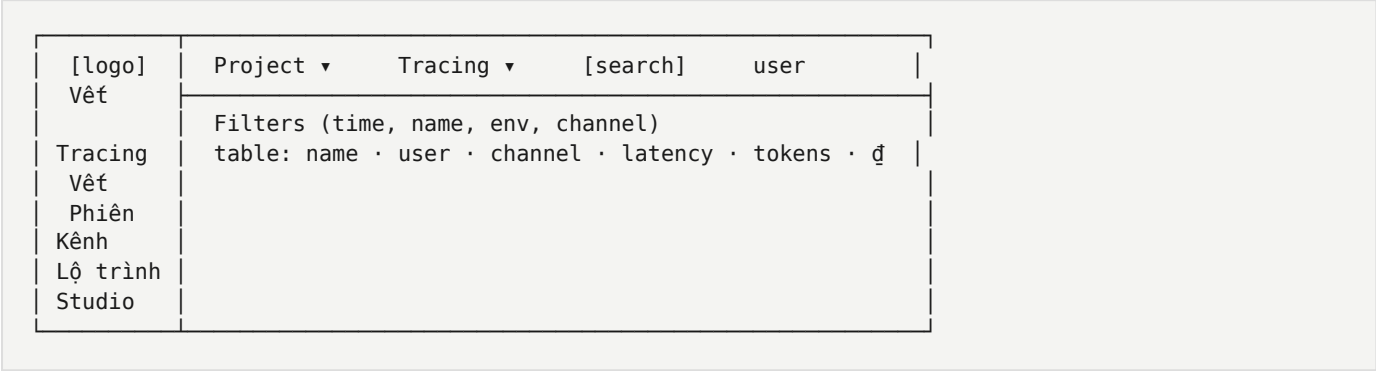
Allowed	Not allowed / avoid
Copy layout, IA, traces table, waterfall, playground from MIT web/ source.	Langfuse wordmark, knot logo, “Langfuse” in chrome, implying official Cloud.
Rebrand logo, product name, accent color, favicon.	Copying ee/ UI (commercial license).
One license/NOTICE: “UI portions derived from Langfuse (MIT), Copyright Langfuse GmbH.”	Shipping an identical Cloud clone as a secret reskin with no new screens.

Practical rule: Langfuse shell + our brand is the traces/playground look. Differentiation is *Kênh, Lộ trình, hội thoại Zalo*. The logo change does not carry the product.

5. How the app looks

Dense ops console (Langfuse / Datadog), not a marketing landing page. Dark sidebar, light or system main pane. Vietnamese labels.

5.1 Shell (copy)



Trace detail (copy Langfuse panel model):

- **Left:** observation tree
- **Center:** timeline / waterfall
- **Right:** input / output / metadata

Desktop: resizable columns. Mobile: tabs — Cây · Timeline · Graph · Chi tiết.

5.2 Brand (change)

- Our mark + wordmark (collapsed sidebar = icon only).
- Accent: pick one (recommend deep green or ink + one highlight, not Langfuse orange, so screenshots differ).
- Favicon, OG image, login screen name.
- Copy: Vết, Phiên, Quan sát, Lần sinh, Chi phí (ước tính), Kênh, Lộ trình.

5.3 New screens (cannot copy)

Hội thoại (session replay) — user-centric, next to the waterfall:

Zalo · user 8477...	14:02 khách: "hủy đơn"	Span: zalo.inbound
	14:02 FPT intent=cancel	Span: fpt.nlu
	14:03 bot: "mã đơn?"	Span: zalo.outbound

Kênh: webhook URL, last event, signature failures, forward target, pause tap.

Lộ trình: graph Zalo → FPT → Claude → Zalo with drop-off counts.

Turn receipt: tokens, ASR ms, TTS voice, Zalo msg_id, estimated cost — Claude /usage feel, per turn.

Empty states: “Chưa có webhook Zalo” with copy-paste hook URL, not “No traces yet.”

6. Information architecture

Theo dõi	
Vết	traces
Phiên	sessions (channel + user)
Quan sát	flat observations
Kênh	
Zalo OA	
Zalo Bot	
FPT.AI Conversation	
Viettel (ASR / TTS / NLP)	
Lark	
Google Chat	
.NET / Teams	
Lộ trình	named pipelines
Studio	
Playground	
Prompt	
Đánh giá	datasets + scores on turns
Cài đặt	
Project, API keys, LLM connections, members	

Playground providers: Anthropic, OpenAI-compatible, FPT AI Factory, **Amazon Bedrock**, **Google Cloud Vertex AI**, **Microsoft Foundry**. Channel connectors (Zalo, Google Chat, Lark) do *not* live in Playground — Vertex ≠ Google Chat, Foundry ≠ Teams.

7. Part A — Langfuse-class core

Feature	v1 bar
Projects + public/secret keys	Create project, ingest auth via header.
Traces / observations / generations	List, filter, detail tree + I/O. Status, latency, tokens.
Sessions	Group by (channel, user_id) or explicit sessionId.
Dashboard	Trace count, error rate, p50/p95 latency, tokens, estimated USD/VND, volume by channel.
Prompts	CRUD + version number. Enough to load in Playground.
Playground	Chat against configured LLM connection. Log as a trace.
Scores / evals	Manual score + one LLM-as-judge on a turn (“bot replied”, “intent != unknown”).
Ingest	HTTP JSON + OTLP HTTP.

Do not build in v1: annotation queues, prompt experiments at Cloud scale, saved views at Langfuse depth, ClickHouse-only analytics, 50 framework integrations.

LLM connections (Playground / judge)

- Anthropic Messages API (refresh model ids; do not ship only Claude 3.x names).
- OpenAI-compatible (LiteLLM, vLLM, OpenRouter, FPT Factory <https://mkp-api.fptcloud.com/v1>).
- Named preset: **FPT AI Factory** with Bearer key + catalog model field.
- **Amazon Bedrock, Google Cloud Vertex AI, Microsoft Foundry** — first-class adapters (next section). Not a hidden “custom OpenAI base URL.”

7a. Built-in hyperscaler support layers

These are **generation providers** (Part A): Playground, LLM-as-judge, SDK wraps, cost/usage on generation spans. They are *not* chat channels (Part B). A Zalo turn may still *call* Bedrock underneath; the span is generation with provider=bedrock.

Langfuse Cloud already lists Bedrock and Vertex as LLM connections. Vết copies that job and adds **Microsoft Foundry** as a named layer (Azure AI Foundry / model deployments), which Langfuse does not treat as a first-class VN-relevant enterprise path next to .NET.

Layer	What it is	What it is not
Amazon Bedrock	Managed models in an AWS account (Claude, Nova, Llama, ...) via Converse / InvokeModel	Not CloudWatch. Not “AWS Chat.”
Google Cloud Vertex AI	Gemini and partner models on a GCP project + location	Not Google Chat (Workspace bot). Not Google AI Studio-only (that is a sibling adapter, API key).
Microsoft Foundry	Azure AI Foundry project: deployments, Azure OpenAI-compatible, Foundry model APIs, Entra or key	Not Teams / Bot Framework inbound (that stays .NET channel). Not “Azure Monitor.”

Every hyperscaler connection stores: provider enum, region or location, project/account id, deployment or model id, auth ref (never shown in traces), extra headers. Generations record provider, model, region, latency, token usage, estimated cost.

Amazon Bedrock

- **APIs:** Converse and ConverseStream first (tool use, system, inference config). Fallback InvokeModel / InvokeModelWithResponseStream for models that are not on Converse.
- **Auth:** AWS access key + secret + optional session token, *or* default credential chain on the server (IAM role). Optional Bedrock API key if the account uses it. Region is required (e.g. us-east-1, ap-southeast-1).
- **Model ids:** first-class list plus custom: anthropic.claude-*, amazon.nova-*, meta.llama-*, mistral.*, cohere.*. Cross-region inference profiles allowed as custom model names.
- **SDK wrap:** wrapBedrock(client) around AWS SDK v3 BedrockRuntimeClient (JS) and boto3 bedrock-runtime (Python). Map usage from Converse usage to our token fields. Stream: TTFT if cheap.
- **Playground / judge:** same adapter; require bedrock:InvokeModel and bedrock:InvokeModelWithResponseStream (Langfuse’s documented IAM pair).
- **Optional later:** Bedrock Guardrails as metadata on the span, not a separate product.

- **Cost:** published Bedrock token rates by model id, labeled ước tính; cache tokens if the model returns them.

Google Cloud Vertex AI

- **API:** Vertex Gemini generateContent / stream (v1). Location + project are required (e.g. asia-southeast1 for VN-adjacent, or us-central1).
- **Auth:** service account JSON (uploaded once, stored encrypted) or Application Default Credentials on the server. Do not put the JSON in traces.
- **Model ids:** gemini-2.5-*, gemini-2.0-*, gemini-1.5-*, plus custom publisher models enabled on the project.
- **SDK wrap:** wrapVertex(client) around @google-cloud/vertexai / official Python Vertex SDK. Record prompt + candidates, safety ratings as metadata, token counts from usageMetadata.
- **Sibling, not a substitute:** Google AI Studio (generativelanguage.googleapis.com + API key) may share the Gemini schema. Ship as provider google-ai-studio if it is a one-file adapter; Vertex remains the enterprise default.
- **Do not** reuse the Google Chat webhook verifier for Vertex. Different credentials, different objects.

Microsoft Foundry

- **What to support:** Azure AI Foundry *project* endpoint + a *deployment* name. Chat Completions (OpenAI-compat) first; enable Responses API flag when the deployment requires it (same pattern Langfuse uses for some OpenAI-compat gateways).
- **Auth:** API key on the Foundry/Azure OpenAI resource, or Microsoft Entra ID (DefaultAzureCredential) — required for many enterprise tenants. .NET must use Azure.Identity; JS/Python equivalent.
- **SDK wrap:** wrapFoundry(client) — Azure OpenAI client (AzureOpenAI in Python, @azure/openai or OpenAI client with Foundry base URL). Record deployment, api-version, tokens, latency.
- **.NET:** same wrap plus OTLP so Foundry calls inside a Teams / Agents SDK turn nest under channel.inbound when the bot is on Microsoft 365.
- **Playground:** connection type “Microsoft Foundry” with fields: endpoint, deployment, api-version, auth mode (key vs Entra). Model dropdown = deployments the user lists, not a hardcoded OpenAI list only.
- **Not in this adapter:** Copilot Studio export, Azure Monitor. Foundry is the model plane; Teams is the channel plane.

Shared adapter contract

```
interface LlmAdapter {
  id: "anthropic" | "openai-compat" | "fpt-factory"
    | "bedrock" | "vertex" | "foundry" | "google-ai-studio"
  chat(req): AsyncIterable or Message // playground + judge
  wrap(sdkClient): TracedClient       // application SDK
  parseUsage(response): TokenUsage
  estimateCost(usage, model): number | null
}
```

Judge evals need tool-calling on the adapter (Langfuse already requires this for OpenAI-compat judges). Bedrock Converse and Vertex Gemini function-calling must be implemented before that provider can run LLM-as-judge. Until then, judge falls back to Anthropic or OpenAI-compat.

UI: Cài đặt → Kết nối LLM → cards for each provider (logo, region, last test ping). Test button: one cheap completion, store latency, do not log the secret.

8. Part B — Vietnamese routes & channels

Each connector: verify → normalize to a **turn** → optional forward. Same span names everywhere.

Span	Meaning
channel.inbound	User message from Zalo / Lark / Chat / Bot Framework
nlu	FPT intent, Viettel NER/segment/spell
generation	Claude, Bedrock, Vertex, Foundry, FPT Factory, vLLM, etc.
tool	CRM, search, booking
speech.asr / speech.tts	Viettel STT/TTS
channel.outbound	Bot reply on the same channel

Session = (channel, user_id) e.g. zalo:oa_123:user_456.

Route = named pipeline e.g. zalo-oa → fpt-conversation → claude → zalo-reply.

8.1 Zalo (ship first)

Two official surfaces:

- **Official Account:** POST webhook, X-ZEvent-Signature = sha256(appId + data + timestamp + OA secret). Events: user_send_text/image/file/..., oa_send_*, follow/unfollow, GMF groups.
- **Zalo Bot:** X-Bot-API-Secret-Token, message.text.received, etc.

Endpoint: POST /hooks/zalo/{oa|bot}/{projectId}

1. Verify MAC / token.
2. Emit channel.inbound.
3. Optional **forward** to customer bot URL (transparent tap) with timeout and kill switch.
4. Match oa_send_* as channel.outbound via msg_id / quote_msg_id.

This is the default VN CX path.

8.2 FPT.AI Conversation / Agents

- Send to bot: POST https://bot.fpt.ai/api/get_answer/ (channel, app_code, sender_id, message).
- Bot replies to *your* webhook (BotManager.parse_response).
- FPT Agents also send channel + broker_id (Zalo OA id, Facebook Page, Teams, ...).

Wrap get_answer and/or ingest the response webhook so one trace is: user text → FPT DME → reply payloads (buttons, carousel). Tag route=fpt-conversation; nest channel=zalo when FPT is only the brain.

FPT AI Factory (mkp-api.fptcloud.com/v1) is a **generation** provider, not a channel.

8.3 Viettel AI

Public APIs are speech + NLP, not a Zalo-style chat webhook:

- ASR `.../voice/api/asr/v1/rest/decode_file` (+ WebSocket streaming)
- TTS `.../voice/api/tts/v1/rest/syn`
- NLP: spell-check, NER, POS, segment on `viettelgroup.ai/nlp/api/v1/...`

SDK wrappers: each call is `speech.asr`, `speech.tts`, `nlu.*` with duration, voice id. Cyberbot/callbot: no self-serve webhook in this plan — instrument inside the customer's adapter via .NET/Python.

Langfuse has no ASR/TTS observation type. Add it.

8.4 Lark / Feishu

- Event `im.message.receive_v1`.
- Webhook (challenge + optional encrypt) or SDK WebSocket long-connection.

POST `/hooks/lark/{projectId}` — echo challenge, decrypt if configured. Domain switch: Lark vs Feishu. Normalize to the same turn schema.

8.5 Google Chat

Google Workspace Chat apps: HTTP endpoint, event payload, verification. Space/DM + thread id → session. Do not confuse with Gemini (generation provider).

POST `/hooks/google-chat/{projectId}`

8.6 .NET inbound

Not a messenger. C# / ASP.NET is the default at many FPT / Viettel / bank shops. Langfuse is Python/JS-first; that is a hole.

Ship:

- NuGet tracer: `ActivitySource` → OTLP to Vết.
- ASP.NET middleware: start a turn on HTTP POST (Bot Framework activity or forwarded Zalo webhook).
- Map Microsoft 365 Agents / Teams SDK spans (`agents.adapter.process`, turn metrics) into our tree.
- Optional channel tag `msteams`.

8.7 Later, same adapter interface

Facebook Messenger, Instagram, Livechat, Viber, Telegram. Do not block v1.

VietAPI and similar gateways: document as OpenAI/Anthropic-compatible generation presets, labeled as gateways, not labs.

9. Canonical data model

```
Project
  id, name, publicKey, secretKeyHash

ChannelConfig
  projectId, type (zalo_oa | zalo_bot | fpt | viettel | lark | gchat | custom)
  secrets (encrypted), webhookPath, forwardUrl, enabled

Route
  projectId, name, steps[] // display + filtering

Trace
  id, projectId, name, sessionId, userId
  channel, routeId, release, environment, tags[]
  start, end, status, metadata

Observation (span)
  id, traceId, parentId, type
  generation | span | tool | channel.inbound | channel.outbound
  | nlu | speech.asr | speech.tts
  name, start, end, status, input, output, model, usage, metadata

Session
  id, projectId, channel, userId, firstSeen, lastSeen

Score
  traceId or observationId, name, value, comment, source (manual|eval)

Prompt
  projectId, name, version, messages[], config
```

Usage on generations: input_tokens, output_tokens, cache read/create, estimated cost (labeled ước tính). Viettel spans: audio ms, voice id, not token USD unless the user sets a rate card.

Store payloads as JSON (redact PII fields by config). Optional object store for ASR audio — off by default.

10. Ingest modes

```
Mode 1 – Tap (zero bot rewrite)
  Zalo →webhook→ Viettel →forward→ customer bot
                        |
                        └─ trace inbound / outbound

Mode 2 – SDK (in-process)
  wrapAnthropic / wrapBedrock / wrapVertex / wrapFoundry / wrapOpenAI / wrapFptGetAnswer /
  wrapViettelTts
  → POST /api/ingest or OTLP

Mode 3 – OTLP
  .NET / Teams SDK / any OTel → /otlp/v1/traces
```

Zalo / Lark / Google Chat: Mode 1 first. FPT / Viettel / Anthropic / Bedrock / Vertex / Foundry: Mode 2. .NET: Mode 2 + 3 (Foundry calls often live here).

Forwarding is a production liability (latency, dual delivery, secrets). v1: timeout, retries=0 or 1, kill switch, log forward failures as spans. SDK path is the reliable default once the bot can add a package.

11. Architecture & repo layout

apps/web	Next.js App Router – VI UI (shadcn)
apps/ingest	Webhooks, signature verify, optional forward
packages/schema	Zod: turns, spans, channel enums
packages/sdk-js	observe, wrapAnthropic, wrapBedrock, wrapVertex, wrapFoundry, wrapOpenAI
packages/sdk-py	same
packages/sdk-dotnet	OTLP + ASP.NET middleware + Azure.Identity for Foundry
connectors/	
zalo-oa, zalo-bot, fpt-conversation, viettel-speech, lark, google-chat	
packages/llm-adapters	
anthropic, openai-compat, fpt-factory, bedrock, vertex, foundry	
LICENSE, NOTICE	MIT + Langfuse MIT attribution if UI source is copied

Stack: Next.js, TypeScript, Tailwind, shadcn/ui. Postgres for traces (SQLite only for a laptop demo if Postgres is painful — do not design for SQLite in production). Redis optional (forward queue). No ClickHouse until one project is huge.

UI strategy:

1. Preferred: vendor Langfuse MIT components for traces/table/waterfall into apps/web, rebrand, add new routes.
2. If vendoring the shell is too heavy tonight: rebuild the IA with shadcn (sidebar, DataTable, resizable panels) and match Langfuse layout, not pixels.

Either way, new pages (Kênh, Lộ trình, hội thoại) are original.

12. APIs

Endpoint	Role
POST /api/ingest	SDK traces (batch of observations)
POST /otlp/v1/traces	OTLP HTTP
GET /api/traces, GET /api/traces/:id	UI
GET /api/sessions, GET /api/stats	List + dashboard
POST /hooks/zalo/oa/:projectId	OA webhook
POST /hooks/zalo/bot/:projectId	Bot webhook
POST /hooks/fpt/:projectId	FPT bot response webhook
POST /hooks/lark/:projectId	Lark/Feishu events + challenge
POST /hooks/google-chat/:projectId	Google Chat
POST /api/playground	Server-side LLM call, then ingest

Auth: ingest uses public key id + secret (or Bearer). Webhooks use platform signatures, not the project secret in query string. No SSO in v1.

13. SDKs

JavaScript / TypeScript

```
import Anthropic from "@anthropic-ai/sdk"
import { wrapAnthropic, observe } from "@vet/sdk"

const client = wrapAnthropic(new Anthropic(), { publicKey, secretKey, baseUrl })

await observe("hồ-trợ-khách", async () => {
  return client.messages.create({ model, max_tokens, messages })
}, { sessionId, userId, tags: ["zalo"] })
```

Must capture: `messages.create`, `streaming`, `tool_use` child spans, usage including cache, errors. Never send `ANTHROPIC_API_KEY` to Vết.

Same pattern for hyperscalers: `wrapBedrock`, `wrapVertex`, `wrapFoundry` — never send AWS/GCP/Azure secrets to Vết ingest, only traces. Must capture: `model/deployment`, `region` or `location`, `usage`, `latency`, `errors`. Streaming where the official SDK supports it.

Python

Same: `wrap_anthropic`, `observe`, FPT `get_answer` helper, Viettel REST helpers.

.NET

OTLP exporter + `VetTracing.UseAspNet()`. Sample ASP.NET bot that accepts a fake Zalo JSON body and records a turn.

Demo without vendor keys: seed 10–15 Vietnamese traces (Zalo cancel-order, FPT intent, tool loop, Viettel TTS, Anthropic error, Lark DM).

14. Security & PII

- Zalo/Lark payloads contain names, user ids, sometimes phones. Default: store; settings: redact keys matching phone, email, address.
- Webhook secrets encrypted at rest. Hook URLs are unguessable (`projectId` + random suffix).
- Do not log raw OA secret. Do not forward to a third-party Langfuse Cloud.
- Audio (Viettel ASR): opt-in retention; default drop binary after transcription span is written.
- Playground keys stay server-side.

15. Implementation sequence

Do this in order. Do not start Lark before Zalo + FPT.

Slice 0 — Tonight bootstrap

1. Scaffold Next.js + shadcn at repo root (or apps/web). Uncommon port (e.g. 43173).
2. Vietnamese layout: sidebar IA from section 6, our logo placeholder, `lang="vi"`.
3. In-memory or SQLite/Postgres: Trace + Observation.

4. Seed Vietnamese traces including a fake Zalo turn.
5. Pages: Tổng quan, Vết list, Vết detail (tree + I/O). Empty/loading/error.
6. POST /api/ingest + JS observe + wrapAnthropic (create first; stream if time).
7. README: how to run, MIT NOTICE if any Langfuse files were copied.

If Anthropic key missing: playground shows a Vietnamese message; seeded traces still demo the UI.

Slice 1 — Zalo OA tap

1. Channel config UI: app id, OA secret, forward URL.
2. Signature verify + inbound/outbound matching.
3. Forward with timeout + disable switch.
4. Hội thoại pane on session detail.

Slice 2 — FPT.AI Conversation

Wrap `get_answer` and response webhook; nest under the Zalo turn when `sender_id` matches.

Slice 3 — .NET + OTLP

NuGet + sample ASP.NET; map one activity to a turn.

Slice 4 — Viettel wrappers

ASR/TTS/NLP spans on the same turn.

Slice 5 — Lark + Google Chat

Same connector interface as Zalo.

Slice 6 — Prompts, Playground, turn evals

FPT Factory as named LLM connection; cost table (ước tính). Judge may use Anthropic or OpenAI-compatible until hyperscaler tool-calling is ready.

Slice 7 — Hyperscaler adapters (Bedrock, Vertex, Foundry)

1. LLM connection UI cards + encrypted secrets + "test ping."
2. Playground chat via each adapter (non-stream first, then stream).
3. SDK wraps: `wrapBedrock`, `wrapVertex`, `wrapFoundry` — generations tagged provider + region/project/deployment.
4. Tool-calling on Converse / Gemini / Foundry enough for LLM-as-judge, or keep judge on Anthropic until then.
5. .NET sample: Foundry deployment inside one Agents SDK turn, visible as nested generation.

Do not start Slice 7 before Slice 0 traces work. Do not block Zalo on AWS credentials.

16. Acceptance checks

Core is done when:

- Sidebar + traces list + trace detail work in Vietnamese on desktop and a narrow viewport.

- Ingest creates a visible trace without restarting the server.
- Seeded Zalo-shaped turn shows inbound + generation + outbound.
- Anthropic wrap records model, tokens, latency when a key is present; degrades cleanly when not.

Hyperscaler layers are done when:

- A Bedrock, Vertex, and Foundry connection can each complete a Playground message and show a generation with provider + region/location/deployment.
- Invalid credentials fail the test ping without creating a successful trace.
- Google Chat webhooks still work without Vertex credentials (and the reverse).

Channel layer is done when:

- A signed fixture POST to `/hooks/zalo/oa/...` appears as a session, not only a raw log line.
- Invalid signature is 401 and does not create a turn.
- Forward can be turned off without breaking ingest.

17. Risks & kill criteria

Risk	Mitigation
Looks like a Langfuse sticker swap	Ship hội thoại + Kênh in the first screenshots, not only traces.
Vendoring Langfuse web is a week of deps	Tonight: shadcn clone of the IA, not a full vendor.
Webhook forward dual-sends or hangs the OA	Short timeout, kill switch, prefer SDK.
FPT/Viettel payloads change	Versioned parsers per connector.
Cyberbot never gets a public API	Do not block the roadmap.
Scope explosion	If Slice 0 + Zalo signature verify are not done, stop adding Lark <i>and</i> stop adding Bedrock/Vertex/Foundry.
IAM / Entra / GCP ADC in a laptop demo	API keys or pasted keys for ping; document role-based auth for production. Never put JSON keys in traces.
Confusing Vertex with Google Chat, Foundry with Teams	Separate nav: LLM connections vs Kênh. Copy in UI: “Mô hình (Vertex)” vs “Kênh Google Chat”.

Kill / cut: If the first merged UI is only an English traces table, the plan has failed even if ingest works. Vietnamese chrome + one Zalo-shaped session is the bar.

18. Appendix — connector notes

Zalo OA signature

mac = sha256(appId + data + timeStamp + 0AsecretKey) with data the JSON body string as documented. Prefer X-ZEvent-Timestamp when present. 7-day CS message window is a Zalo product limit, not ours —

document it.

FPT get_answer

channel is often "api" for custom integrations; production OA traffic may already be connected inside FPT's console. We still want the response webhook so replies are traced even when FPT owns the Zalo pipe.

Viettel TTS example fields

text, voice, speed, tts_return_option (stream / wav / mp3). Span output: duration, format, not the full audio blob by default.

Lark challenge

Return { "challenge": "<value>" } within 1s on URL verification. Encrypted events need the encrypt key from the developer console.

Google Chat

Verify per Google Chat app docs (audience / bearer). Map space.name + user to session id. This is *not* Vertex AI.

Amazon Bedrock

Use Converse in the configured region. Model id is the Bedrock id (optionally inference profile). IAM: bedrock:InvokeModel, bedrock:InvokeModelWithResponseStream. Do not send AWS secrets to the browser; Playground calls go server-side.

Google Cloud Vertex AI

Client needs project, location, model resource name. Auth: service account with Vertex User (or equivalent) on that project. asia-southeast1 is the default suggestion in VN docs, not a hardcoded. Distinct from Google AI Studio API keys.

Microsoft Foundry

Endpoint looks like an Azure AI Foundry / Azure OpenAI resource URL. Required: deployment name + api-version. Auth: key or Entra. OpenAI-compatible /chat/completions first; Responses API toggle if the deployment needs it. Pair with the .NET OTLP path for Teams bots that call Foundry.

Langfuse UI files to study if vendoring

- Sidebar / nav (collapsible, project switcher)
- Traces table + filter rail
- trace2 layout: tree, timeline, I/O, mobile tabs
- Playground

Do not vendor worker, ClickHouse migrations, or ee/.

Document control

This PDF is the implementation source of truth for the next build. Update it if you cut Slice 5 or 7, or rename the product. Excel eval and a generic usage-report SaaS stay out of scope until this core + Zalo + FPT path exists.

Working title Vết · 9 Sep 2026 (rev. hyperscalers) · For implementation, not publication as Langfuse.